CS6886 Assignment 5

Q1. My machine

Table 1 lists my laptop's specification (also in `machine_spec.txt`).

Table 1: Machine specification

Field	Value
CPU model	AMD Ryzen 7 6800H (Zen 3+, Rembrandt)
Physical / logical cores	8 / 16 (2 threads per core)
Performance / efficiency cores	N/A: 8 identical cores, not a hybrid design
Base clock	3.2 GHz
Maximum single-core clock	4.75 GHz
All-core clock under load	3.85 GHz (measured, AVX2 MAC load on 8 cores)
SIMD instruction set and width	AVX2 + FMA, 256 bits (8 FP32 values)
MAC units per core	2, each 256 bits wide
Vector registers	16 YMM registers (256 bits)
L1 data cache	32 KB per core
L2 cache	512 KB per core, not shared
L3 cache	16 MB, shared by all 8 cores
Cache line size	64 bytes
Memory	16 GB DDR5 SODIMM, 4800 MT/s (rated 5600), 1 channel
Operating system	Fedora Linux 43, kernel 7.2.5
Compiler	clang 21.1.8

A MAC (multiply-accumulate) computes $a \times b + c$ in one instruction (FMA in AVX2). Each core has 2 MAC units, each works on 8 floats at once, and one MAC counts as 2 operations.

$$\begin{aligned}
\text{Operations per cycle per core} &= 2 \times 8 \times 2 = 32 \\
\text{Peak, one core} &= 32 \times 4.75 \text{ GHz} = 152.0 \text{ GFLOP/s} \\
\text{Peak, all cores} &= 32 \times 8 \times 3.85 \text{ GHz} = 985.6 \text{ GFLOP/s} \\
\text{Memory bandwidth} &= 4800 \text{ MT/s} \times 8 \text{ bytes} \times 1 \text{ channel} = 38.4 \text{ GB/s} \\
\text{Ridge point, one core} &= 152.0/38.4 = 3.96 \text{ FLOP/byte} \\
\text{Ridge point, all cores} &= 985.6/38.4 = 25.67 \text{ FLOP/byte}
\end{aligned}$$

I count 8 cores, not 16 threads, since two threads on a core share its MAC units. Table 2 checks these limits.

Table 2: Quick tests of the calculated limits

Test	Result
Maths only, one core	About 145 GFLOP/s (95% of peak), at 4.65 GHz
Maths only, 8 cores	About 945 GFLOP/s (96% of peak), at 3.85 GHz
Reading memory, 8 threads	29.3 GB/s (76% of the calculated 38.4 GB/s)
Reading memory, 1 thread	28.2 GB/s

Under full load the chip reached 96 °C and the clock fell to 3.85 GHz, so I use that for the all-core peak. With one memory channel, one core already uses almost all the bandwidth.

Q2. Naive baseline

Times are the median of repeated runs (11 per variant, 5 for naive), built with the given Makefile.

Table 3: Naive baseline

N	d	Time (ms)	GFLOP/s	% of one-core peak
1024	64	90.0	2.98	2.0%
2048	64	382.8	2.81	1.8%
4096	64	1,576.3	2.72	1.8%
4096	128	3,659.6	2.35	1.5%

Work grows as N^2d: doubling N makes it about 4× slower, doubling d about 2.3×. At N=4096, d=64, $QK^\top$ takes 35% of the time, softmax 4% and the multiply by V 62%.

The matrix multiplications handle one number at a time: each adds into a single running total, so the compiler cannot vectorise them, and each MAC waits about 4 cycles for the one before. That allows about 2.3 GFLOP/s, matching the 2.2 GFLOP/s measured for the V step. The limit is this waiting, not memory.

Q3. Optimisations

Table 4: Single-threaded variants; speedup is compared with naive

Variant	N	d	Time (ms)	GFLOP/s	% of peak	Speedup
loops	1024	64	11.8	22.8	15.0%	7.6×
tiled	1024	64	5.64	47.6	31.3%	16.0×
fused	1024	64	5.81	46.2	30.4%	15.5×
best	1024	64	2.79	96.1	63.3%	32.2×
loops	2048	64	48.6	22.1	14.5%	7.9×
tiled	2048	64	23.5	45.7	30.1%	16.3×
fused	2048	64	22.9	46.9	30.8%	16.7×
best	2048	64	10.8	99.5	65.5%	35.5×
loops	4096	64	237.9	18.1	11.9%	6.6×
tiled	4096	64	131.7	32.6	21.5%	12.0×
fused	4096	64	90.9	47.3	31.1%	17.3×
best	4096	64	41.8	102.7	67.5%	37.7×
loops	4096	128	371.2	23.1	15.2%	9.9×
tiled	4096	128	175.6	48.9	32.2%	20.8×
fused	4096	128	138.9	61.8	40.7%	26.3×
best	4096	128	79.4	108.2	71.2%	46.1×

All 20 `make check` comparisons pass.

(a) Loop reordering: `attn_loops.c`

Transposing K and reordering the loops lets the inner loop read memory in order with no single running total, so the compiler vectorises it.

What limits it now: every row of Q reads all of K and V again. At N=4096 they are 1 MB each, too big for the 512 KB L2 cache, so they come from the slower L3 every time and speed falls from 22 to 18 GFLOP/s. Softmax also still works on one number at a time.

(b) Tiling: `attn_tiled.c`

I compute 6×16 output tiles in registers with AVX2, on blocks of K and V small enough to stay in cache.

What limits it now: the matrix maths is fast, but two other costs remain at N=4096. Softmax still works on one number at a time and takes 55 of the 132 ms. The N×N matrix is 64 MB, too big for any cache, so creating it and moving it to and from RAM costs about 42 ms more.

(c) Fusing: `attn_fused.c`

I take 6 rows at a time through all three steps, so the N×N matrix is never stored. This is 1.45× faster than tiled at N=4096, with no gain at smaller N, where the matrix fit in cache.

What limits it now: with the big matrix gone, softmax is the slow part. It still computes one exponential at a time and takes 60% of the time.

(d) Best version: `attn_best.c`

Three changes to fused: blocks of 48 rows × 128 keys keep the scores in cache; a running maximum and sum per row let each row be done in parts; and my own exponential works on 8 numbers at once instead of `expf`. This reaches 63–71% of peak.

What limits it now: softmax is down to 13% of the time. Matrix multiplication is 84% and runs at 124 GFLOP/s, 81% of the core's peak, so the CPU's maths units are now the limit and little speed is left to gain.

Q4. OpenMP

Blocks of rows are independent, so I split them across threads, each with its own buffer.

Table 5: OpenMP scaling. Speedup is against 1 thread; GFLOP/s is in `results.csv`

	N=1024, d=64			N=2048, d=64			N=4096, d=64			N=4096, d=128		
Threads	ms	Speedup	Eff.	ms	Speedup	Eff.	ms	Speedup	Eff.	ms	Speedup	Eff.
1	2.51	1.00×	100%	10.1	1.00×	100%	41.8	1.00×	100%	79.3	1.00×	100%
2	1.81	1.39×	69%	5.10	1.99×	99%	22.4	1.87×	93%	42.7	1.86×	93%
4	1.04	2.42×	60%	3.64	2.79×	70%	13.8	3.03×	76%	26.6	2.98×	74%
8	0.61	4.09×	51%	1.97	5.16×	64%	7.31	5.72×	72%	14.3	5.54×	69%
16	0.54	4.64×	29%	1.86	5.46×	34%	7.88	5.30×	33%	15.5	5.13×	32%

8 threads give 5.7×, not 8×, mainly because of heat: the clock falls from 4.65 to 3.85 GHz, and

even the maths-only test reaches only 6.5×. Memory is not the limit, since K and V stay in cache. N=1024 runs for only 0.6 ms, so start-up costs matter more. 16 threads do not help, since two threads on a core share its MAC units.

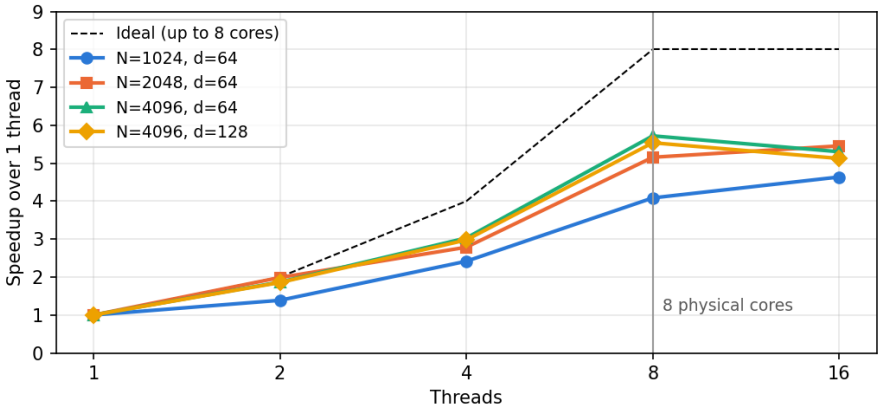


Figure 1: OpenMP speedup against threads; dashed line is ideal

Q5. Roofline

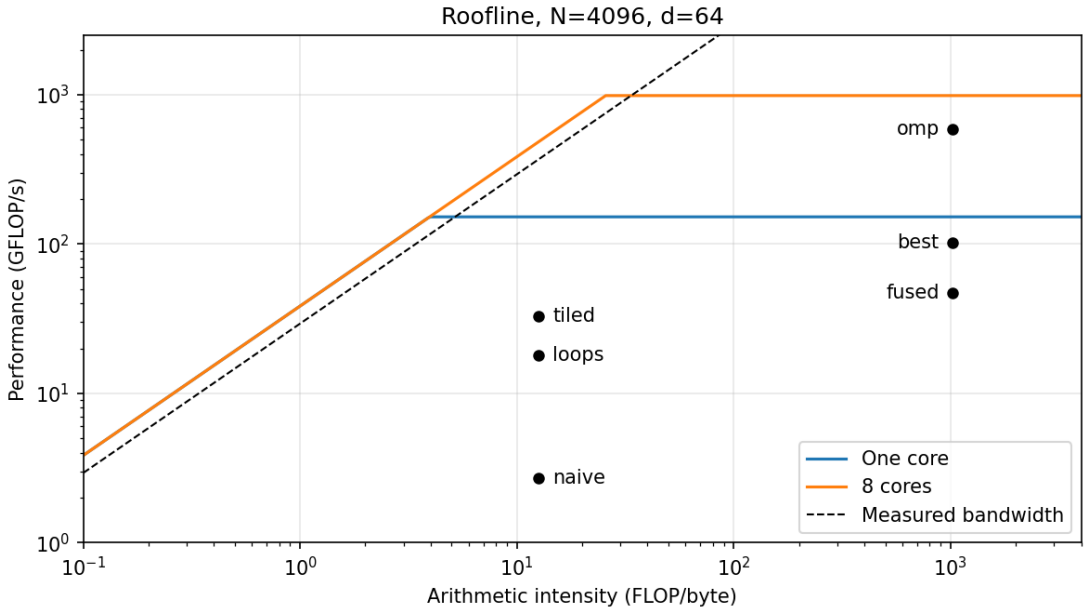


Figure 2: Roofline of my machine with every variant at N=4096, d=64

Arithmetic intensity is $4N^2d$ operations divided by my estimate of the bytes moved to and from RAM. Every variant moves Q, K, V and O ($16Nd$ bytes). naive, loops and tiled also store the 64 MB N×N matrix, which does not fit in L3. I count five passes over it (two to write it, since the CPU reads memory before writing, two for softmax, one for the last step), adding $20N^2$ bytes. This gives 12.64 FLOP/byte with the matrix and 1024 without it.

Table 6: Roofline results (N=4096, d=64)

Variant	FLOP/byte	GFLOP/s	Roof	% of roof	What limits it
naive	12.64	2.7	152.0	2%	One number at a time
loops	12.64	18.1	152.0	12%	Too many memory reads, slow softmax
tiled	12.64	32.6	152.0	21%	Slow softmax, big N×N matrix
fused	1024.00	47.3	152.0	31%	Slow softmax
best	1024.00	102.7	152.0	68%	CPU maths speed
omp (8t)	1024.00	587.7	985.6	60%	CPU maths speed, lower clock

All variants are right of the one-core ridge point (3.96), so RAM does not limit them on one core. naive, loops and tiled move the same data, so they sit in a vertical line. Fusing matters for threads: at 12.64 FLOP/byte a threaded tiled version would fall below the 8-core ridge point (25.67) and be limited by RAM; omp is not.

Time per variant

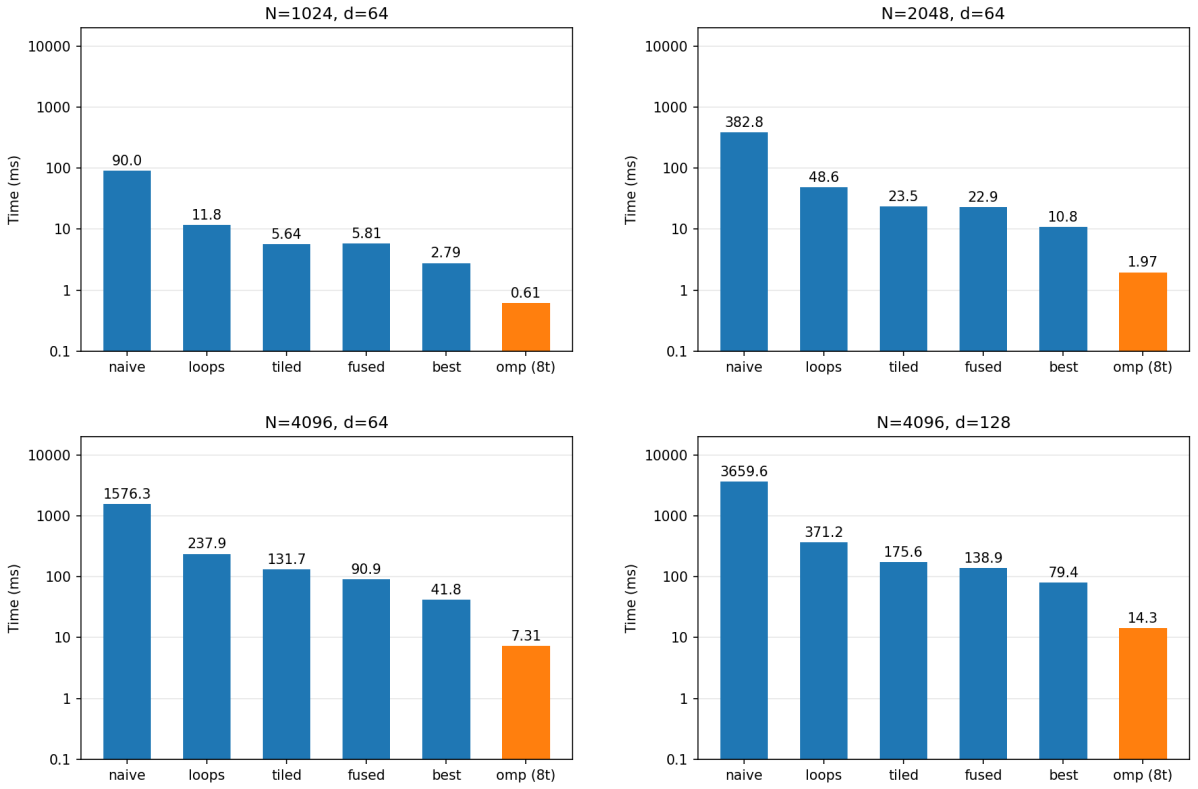


Figure 3: Time of every variant for each input (log scale)